



Panduan Hemat Token untuk Vibecoder

Panduan praktis membangun aplikasi dengan AI Agent — murah, cepat, dan backend tetap aman

Konteks: Project Ai Teacher Assistant (Klassa) · Berdasarkan data konsumsi aktual story 3–5 · Sep 2026

0 · Pahami Dulu: Dari Mana Token Terbuang?

Setiap giliran (turn) percakapan dengan AI agent, **seluruh riwayat konteks dikirim ulang** ke model. Makin panjang sesi, makin besar konteks yang dikirim ulang. Ada 3 jenis token dengan harga berbeda:

Jenis	Artinya	Harga relatif
Input fresh	Token baru yang belum pernah diproses	Mahal (acuan 10x)
Cache read	Bagian konteks yang sama dengan turn sebelumnya, diambil dari cache server	Murah (~1x)
Output	Token yang model generate (jawaban, kode, tool call)	Paling mahal per token

✅ Fakta menenangkan

Cache read yang besar itu **tanda bagus**, bukan pemborosan. Data aktual project: satu sesi build menyerap 78 juta cache read tapi biayanya cuma ~\$2,6. Kalau token itu ditagih sebagai input biasa, biayanya bisa 10x lipat.

❌ Musuh sebenarnya: cache miss & sesi panjang

(1) **Cache miss** — konteks berubah di tengah sesi (edit file besar, campur topik), seluruh konteks dihitung ulang sebagai input mahal. (2) **Sesi panjang** — makin lama sesi, makin banyak konteks terbawa di setiap turn. Dua hal ini yang dikendalikan panduan ini.

📌 Data pembandingan dari project ini

Sesi yang diawali perintah "*analisa mendalam dan menyeluruh*" konsisten menyerap **20–70 juta token** cache read. Sesi yang diawali path file spesifik (review lens) cuma **40–170 ribu**. Perbedaan prompt saja: seratus kali lipat.

1 · Aturan Emas: 1 Story = 1 Sesi

Sesi monster (78 juta token) terjadi karena satu sesi mengerjakan banyak hal: cek progress → hardening → build → fix. Setiap giliran, semua konteks itu terbawa.

✓ LAKUKAN

- 1 story = 1 sesi build. Selesai → commit → tutup sesi.
- "Cek progress", "jalankan server", "review git" → sesi terpisah yang pendek.
- Mulai sesi build langsung dengan path story file.

✗ HINDARI

- Mixing topik dalam satu sesi ("sekalian cek deploy ya Bro").
- Sesi melewati batas 1 hari / 1 story.
- Mengobrol santai di sesi build yang sudah panjang — tiap pesan menambah konteks permanen.

2 · Prompt yang Menghemat 100x

Polanya: SPESIFIK PATH + SCOPE EKSPLISIT

✓ HEMAT (~50 ribu token)

```
Baca _bmad-output/specs/spec-student-portal-auth/stories/5-panel-persetujuan.md, lalu implementasikan bagian batch approve saja. File yang boleh disentuh: src/modules/approvals/** Test: vitest run src/modules/approvals
```

✗ MAHAL (5–70 juta token)

```
"Tolong analisa project ini secara menyeluruh dan mendalam, semua file, lalu tolong perbaiki juga yang kurang"
```

Template prompt build siap pakai

```
Kerjakan story N: [path file story]
Baca HANYA: story file itu + file yang direferensikannya.
Scope: [2-3 kalimat].
Out of scope: [daftar eksplisit yang JANGAN disentuh].
Test: vitest run [modul] + tsc. Full suite JANGAN dulu.
Selesai: ringkas perubahan per file (maks 15 baris)
+ daftar asumsi keamanan yang diambil.
```

💡 Kalimat ajaib untuk backend

Tambahkan selalu di akhir prompt build backend:

"Setelah selesai, tulis daftar asumsi keamanan yang kamu buat + titik yang menurutmu paling berisiko."
Harganya ~500 token output, tapi memancing agent mengoreksi dirinya sendiri sebelum review.

3 · Test Terarah, Bukan Full Suite Tiap Iterasi

Momen	Perintah	Kenapa
Saat iterasi/edit kecil	<code>vitest run src/modules/[modul]</code>	Log pendek, cepat, konteks tak bengkok
Akhir tahap/story	Full suite + <code>tsc --noEmit</code>	Sekali saja, sebagai gerbang DoD
E2E Playwright	Hanya untuk journey DoD	Paling mahal — jangan tiap edit kecil

Log test yang panjang masuk ke konteks dan terbawa sisa sesi. Output test ratusan baris x 20 iterasi = jutaan token sia-sia.

4 · Skalikan Review Lens Sesuai Risiko

Pola 3 review lens (blind hunter / edge-case hunter / verification-gap) bagus — tapi jangan seragam untuk semua story:

Jenis perubahan	Contoh story	Review
Auth, sesi, rate-limit, migrasi DB, akses antar-role	Story 3, 5, 6	3 lens penuh + walkthrough
Agregasi/query baca	Story 7	2 lens: edge-case + verification-gap
UI portal read-only, tampilan	Bagian UI story 4, 7, 8	1 lens: blind hunter (atau cukup verifikasi visual)

✅ Hasil

Hemat 40–60% biaya review tanpa mengorbankan keamanan — story yang kritis tetap dapat perlindungan penuh.

5 · Ambil Keputusan SEBELUM Build

- Keputusan saat blueprint/elicitation = **murah** (diskusi, minim tool call).
- Keputusan di tengah build = **mahal**: agent sudah baca 10 file, asumsi salah → baca ulang → edit ulang → test ulang.
- Contoh nyata: **Open Question ① Story 7** (skor flat vs GradePolicy/KKTP). Putuskan sekarang, tulis di story file — kalau tidak, tambahan ~0,5–1 juta token + risiko rework.

6 · Keamanan Backend: Kunci dengan Invariants, Bukan Prompt Panjang

Jangan minta "pastikan aman ya" (tidak terukur). Yang terbukti bekerja di project ini — daftar invarian pendek di `invoke_dev_with` :

- limiter wajib persisten di DB (bukan in-memory)
- studentId derive dari verifyStudentSession(), parameter identitas dari klien diabaikan
- pesan error generik + dummy-verify agar timing seragam
- PIN_PEPPER fail-fast di production

Invarian pendek & mengikat > prompt keamanan 3 paragraf. Lebih efektif, dan gratis.

7 • Model Tiering

Tugas	Model
Scaffold UI, dokumen, ringkasan, review awal	Model cepat/murah (flash)
Desain migrasi DB, core auth/session, review akhir story kritis	Model kuat — naikkan hanya di sini

8 · Khusus Vibecoder: Aman Tanpa Paham Kode

Vibecoder tidak perlu paham kode untuk aman. Yang perlu dipahami: **proses dan bukti**. Kode itu urusan agent; tugasmu menjaga kontrak dan verifikasi hasil.

8.1 Ganti "baca kode" dengan "tuntut bukti"

Ketika agent bilang "selesai", jawabanmu cuma satu: **"Buktikan."** Minta dia menjalankan aplikasi dan mendemokan alurnya (browser automation bisa klik-klik dan screenshot untukmu). Kamu menilai **perilaku**, bukan kode.

8.2 Lembar uji 5 menit (jalankan sendiri tiap akhir story)

- ☐ Login guru bisa
- ☐ Login siswa NIS + PIN bisa
- ☐ PIN salah 5x → terkunci
- ☐ Buat quiz → siswa kerjakan → nilai muncul di dashboard guru
- ☐ Siswa A tidak bisa lihat data Siswa B
- ☐ Ganti PIN → login pakai PIN baru berhasil

5 menit klik menggantikan 90% kebutuhan memahami kode. Kalau ada yang gagal di checklist, baru agent yang bedah teknisnya.

8.3 Git adalah jaring pengamanmu

✓ Cukup 2 kebiasaan

- **Commit sebelum mulai story** → titik aman; build berantakan = rollback 1 perintah, nol kerugian.
- **1 story = 1 commit** → kalau rusak, tahu persis penyebabnya.

✗ Akibat kalau diabaikan

- Tanpa titik aman, satu build buruk bisa menular ke area yang tadinya baik.
- Commit campur aduk = tidak bisa rollback selektif.

8.4 Kekuatan vibecoder ada di SPEC, bukan kode

- Tulis perilaku di story file dengan bahasa manusia yang presisi — contoh bagus: *"siswa tanpa NIS tidak punya jalur klaim"*.
- Istilah tidak paham? Tanya: *"jelaskan pakai bahasa sederhana + analogi"* — ~500 token, mental model menempel lama.
- Kode boleh tidak dipahami, **syaratnya perilaku terverifikasi (8.1 + 8.2)**.

8.5 Jangan hemat di titik butamu

Area	Bisa verifikasi sendiri?	Strategi
UI portal, tampilan	✅ Klik-klik	Hemat review, verifikasi visual
Alur bisnis (quiz, nilai, persetujuan)	✅ Checklist 5 menit	Review sedang
Auth, sesi, PIN, migrasi DB	❌ Buta total	Jangan hemat — 3 lens + walkthrough wajib

8.6 Prompt penutup sesi build (wajib)

Sekarang jalankan walkthrough: jelaskan apa yang berubah pakai bahasa manusia (bukan istilah teknis), apa yang harus saya coba sendiri untuk memastikan ini jalan, dan apa yang paling berisiko rusak.

Di BMAD sudah ada skill-nya: `bmad-walkthrough` .

9 · Estimasi Token Story 6–8 (Mengikuti Pola Existing)

Story	Beban	Input fresh	Cache read	Output + reasoning
6 — Pengerasan, Rollover TA & Dokumentasi	Ringan	~1,5–2,5 jt	~10–20 jt	~100–150 rb
7 — Student Progress (Gelombang 2)	Sedang	~3–5 jt	~30–60 jt	~150–250 rb
8 — Submission, Materi & Mode Keluarga (G3)	Paling berat	~4–7 jt	~40–80 jt	~200–350 rb
TOTAL	—	~8,5–14,5 jt	~85–165 jt	~0,45–0,75 jt

⚠ Catatan

(1) Biaya aktual ~\$3–7/story dengan model flash + cache hit tinggi. (2) Faktor pembengkak terbesar: elicitation multi-ronde dan sesi build yang membaca spec besar tiap turn. (3) Open Question ① Story 7 belum diputuskan = tambah 0,5–1 juta token. (4) Dengan penerapan panduan ini, target hemat 30–50% dari angka di atas.

10 · Ringkasan Satu Halaman

Prinsip	Aksi konkret
1 story = 1 sesi	Selesai story → commit → tutup sesi. Hal sampingan = sesi terpisah.
Prompt spesifik path	Selalu sebut file + scope + out-of-scope. Hindari "analisa menyeluruh".
Keputusan pra-build	Selesaikan open question saat elicitation, tulis di story file.
Test terarah	vitest per modul saat iterasi; full suite + tsc hanya di akhir tahap.
Review sesuai risiko	3 lens untuk auth/DB; 2 lens untuk query; 1 lens/visual untuk UI.
Invariants, bukan prompt panjang	Daftar aturan keamanan pendek di invoke_dev_with.
Bukti > kode	"Selesai" = harus didemokan jalan. Checklist 5 menit tiap akhir story.
Git disiplin	Commit sebelum story, 1 story = 1 commit.
Tutup dengan walkthrough	Penjelasan bahasa manusia + yang harus dicoba + risiko terbesar.

🎯 Inti semuanya

Biaya terbesar bukan output, tapi **konteks yang terbawa**. Sesi pendek + scope eksplisit = hemat 50%+ tanpa mengorbankan apa pun. Vibecoder yang aman bukan yang paham kode, tapi yang (1) menuntut bukti perilaku nyata, (2) disiplin commit, (3) tahu di mana dia buta dan membayar review persis di titik itu.